

Habib University

Dhanani School of Science and Engineering

Computer Architecture

EE 371 / CS 330 / CE 321 (Fall 2025)

Homework 3

Release Date:

Due Date:

Total Marks:100

Obtained Marks:

Student Name:

Student ID:

Section:

Instructions:

1. This assignment should be done in pairs.
2. All questions should be answered in **black ink only**.
3. Print the assignment, scan the solution, and upload it on HU LMS before the deadline.

Grading Criteria:

1. Your assignments will be checked by instructor/TA.
2. You can also be asked to give a viva where you will be judged whether you understood the question yourself or not. If you are unable to answer correctly to the question you have attempted right, you may lose your marks.
3. Zero will be given if the assignment is found to be plagiarized.
4. Untidy work will result in reduction of your points.

Submission policy:

No submission will be accepted after the instructor releases the solution on HU LMS.

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	✓
CLO 4	<i>Analyze the</i> performance of cache operations	

Question # 1 (Points: 12, 2+2+2+2+2+2, CLO - 3):

A 5-stage pipelined processor has the following stage latencies:

IF	ID	EX	MEM	WB
250ps	180ps	300ps	270ps	150ps

and also consider the inter-stage register delay to be 20ps.

Also, assume that instructions executed by the processor for a program-A are divided as follows:

ALU/Logic	Jump/Branch	Load	Store
40%	10%	25%	25%

i. Compute the clock cycle time for both pipelined and non-pipelined versions.

ii. Determine the total latency for a store instruction in both processors.

iii. If 1200 instructions are executed with no hazards, calculate the speedup achieved.

iv. Now assume a stall occurs every five instructions. What is the new speedup?

- v. Suggest a possible architectural modification to reduce overall cycle time without changing ISA behavior. Calculate the new pipeline clock cycle time.

- vi. Assuming all stages are fully utilized, calculate the utilization of the instruction memory and data memory over 1000 instruction executions.

Question # 2 (Points: 10, 4+4+2, CLO - 3):

In a pipelined CPU, the multiply instruction (mul) requires two cycles in the EX stage (EXa and EXb).

- i. Explain why this situation causes a structural hazard in the pipeline. Use a pipeline timing chart for a sequence containing two consecutive mul instructions to illustrate the conflict. (Assume there is no stall unit.)

- ii. Clearly highlight or annotate the modifications needed to support a two-cycle mul instruction using Figure 4.58 from the textbook. Indicate any new control signals, hardware components, or changes to existing control logic that prevent this hazard.

- iii. Briefly describe how your proposed modifications resolve the hazard.

Question # 3 (Points: 8, 5+3, CLO - 3):

A processor uses a 2-bit saturating counter branch predictor with the following encoding:

Binary	Meaning
00	Strongly Not Taken
01	Weakly Not Taken
10	Weakly Taken
11	Strongly Taken

Initially, the Branch History Table (BHT) contains the entries of the states of four branches A, B, C, and D as:

- A = 10
- B = 00
- C = 11,
- D = 01

During program execution, the following sequence of branch executions and actual outcomes occurs:

Order	1	2	3	4	5	6	7	8	9	10
Branch	A	B	A	C	A	D	C	A	B	10
Actual Outcome	Taken	Not Taken	Taken	Taken	Not Taken	Not Taken	Taken	Taken	Not Taken	Not Taken

- Simulate the branch predictor operation for this sequence. For each branch execution, show:
 - the initial state
 - the prediction made
 - whether it was correct
 - the updated state.

- ii. Count the total mispredictions and compute the misprediction rate.

Question # 4 (Points: 24, (2+5+3+2)x2, CLO - 3):

Define new instructions for RISC-V

- ```
i. maxadd rd, rs1, rs2, rs3 // Reg[rd] = max(Reg[rs1], Reg[rs2]) + Reg[rs3])
ii. cmpadd rd, rs1, rs2, rs3 // if (Reg[rs1] > Reg[rs2]) Reg[rd] = Reg[rs3] + 1; else
 Reg[rd]=Reg[rs3];
```

For both of the instructions:

- [illegible]

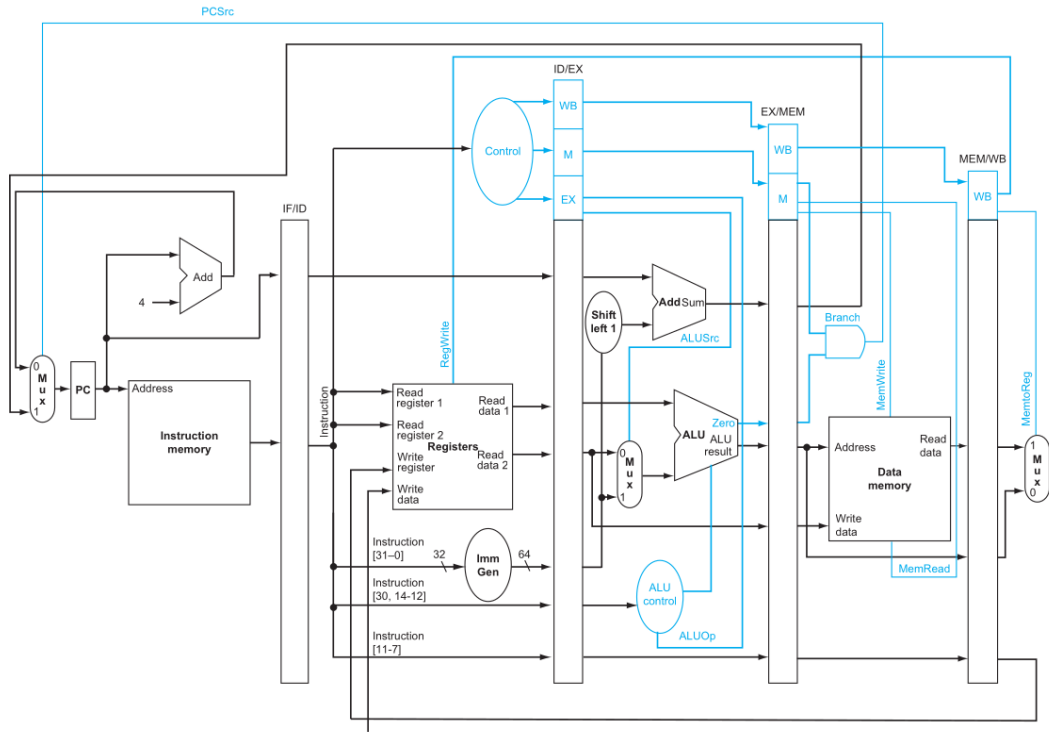
c. What new control signals must be generated?

d. Identify any new forwarding paths needed for a pipelined version.

**Question # 5 (Points: 15, CLO - 3):**

Suppose one of the following control signals in a single-cycle RISC-V processor is stuck-at-1 (always logic 1).

Which instruction(s) would malfunction, what would be the issues observed and why? Refer to the multicycle datapath and control shown in the figure below



i. RegWrite

ii. MemRead

iii. MemWrite

iv. Branch

v. ALUSrc

vi. ALUOp<sub>0</sub>

vii. ALUOp<sub>1</sub>

viii. ALUSrc

ix. MemtoReg

**Question # 6 (Points: 13, 3+4+3+3, CLO - 3):**

The pipelined RISC-V processor executes the following code segment:

```
addi x1, x0, 5 # x1 = 5
addi x2, x0, 10 # x2 = 10
add x3, x1, x2 # x3 = 15
sub x4, x3, x1 # x4 = 10
lw x5, 0(x4) # load word from memory address in x4
add x6, x5, x1 # x6 = x5 + x1
sw x6, 4(x4) # store word from x6 to memory[x4 + 4]
add x7, x6, x3 # x7 = x6 + x3
```

Assume the following initial conditions:

- All registers are initialized to 0 except x0 (which is always 0).
  - Memory location Mem[10] = 40.
- i. Identify all data hazards present in the above sequence (specify the dependent registers and the type of hazard).

- ii. Assuming there is no forwarding and no stall detection, complete a table showing the value of each register (x1–x7) after each instruction finishes execution. Clearly indicate where incorrect values would appear due to hazards.
- iii. Describe how forwarding and hazard detection unit could be used to ensure correct execution of this program. Mention which hazards are resolved by forwarding and which require stalls.



- iv. Redraw the pipeline diagram assuming full forwarding and hazard detection. Clearly label where stalls (if any) occur.

**Question # 7 (Points: 10, CLO - 3):**

The pipelined processor's performance might be better if branches take place during the Decode stage rather than the Execute stage. Show how to modify the pipelined processor to move the branch logic to the Decode stage. How do the stall, flush, and forwarding signals change? You can use a printed version of the figures in the book over which you can draw your suggested modifications.

**Question # 8 (Points: 8, 2+2+2+2, CLO - 3):**

A 64-bit RISC-V pipeline is being redesigned to improve performance by modifying its Execute (EX) stage.

The EX stage is the slowest stage in the pipeline and therefore determines the pipeline clock period, which includes a 20 ps delay for each inter-stage pipeline register.

Each ALU has a propagation delay of 150 ps, and the EX stage dominates all other stage latencies in the pipeline.

The design team plans to introduce new arithmetic instructions that operate on three source registers (e.g.,  $\text{avgadd rd, rs1, rs2, rs3} = ((\text{rs1} + \text{rs2})/2) + \text{rs3}$ ).

To support these multi-operand operations efficiently, three alternative pipeline configurations are considered:

Three configurations are considered:

- Configuration A: Single ALU in the EX stage (baseline design).
- Configuration B: Two ALUs cascaded within the same EX stage (for multi-operand operations).
- Configuration C: EX stage split into two pipeline stages, EX1 and EX2, each containing one ALU.

- i. Compute the pipeline clock cycle time for Configurations A, B, and C.
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
  
- ii. Assuming Register 1, Register 2, and Register 3 represent 64-bit source registers ( $rs_1$ ,  $rs_2$ ,  $rs_3$ ), and ignoring control signals, compute the width (in bits) of the inter-stage pipeline registers for:
  - Configuration B: ID/EXE and EXE/MEM
  - Configuration C: ID/EXE1, EXE1/EXE2, and EXE2/MEM

- iii. Assume both ALUs perform the same operation. Identify and name the control signals required for the ALU stage(s) in Configurations B and C.
- iv. For Configuration B, write the hazard detection and forwarding conditions needed to handle data dependencies between consecutive ALU instructions.  
Use pipeline register notation (e.g., EX/MEM.RegisterRd = ID/EX.RegisterRs1).